SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10 **Total Marks: 100**

ANNEXURE A **Coding Challenge**

Code of Conduct:

*I _**BHARATH SIMHA REDDY(192424361)**_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

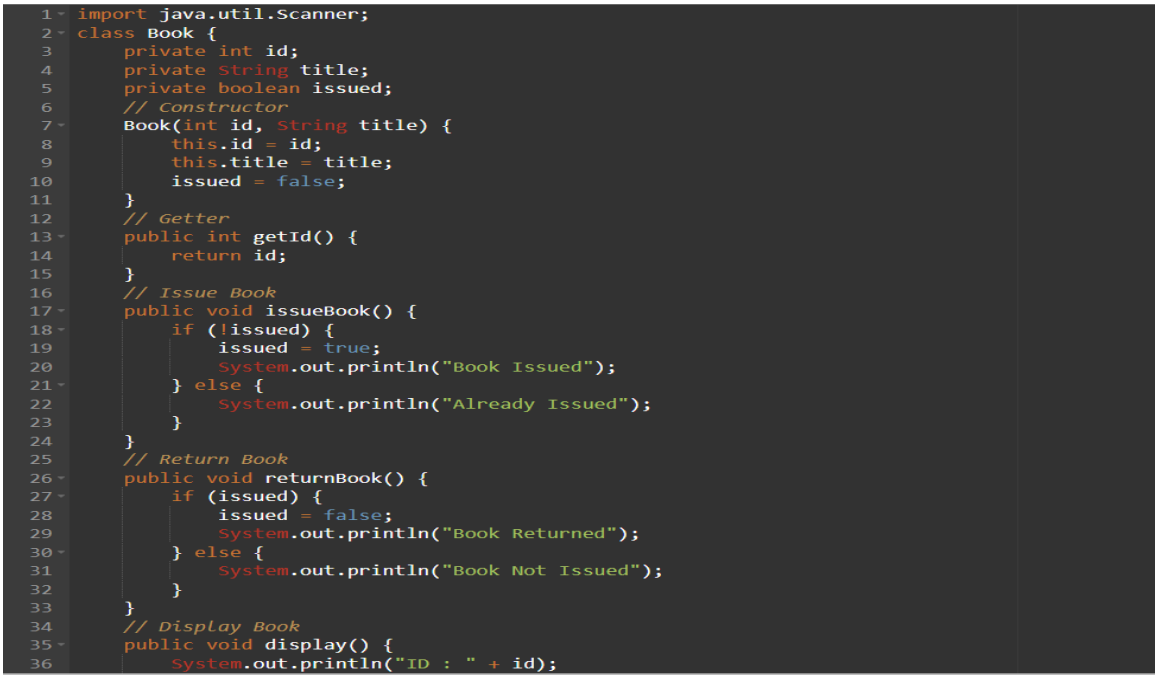
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)



```java
import java.util.Scanner;
class Book {
    private int id;
    private String title;
    private boolean issued;
    // Constructor
    Book(int id, String title) {
        this.id = id;
        this.title = title;
        issued = false;
    }
    // Getter
    public int getId() {
        return id;
    }
    // Issue Book
    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued");
        } else {
            System.out.println("Already Issued");
        }
    }
    // Return Book
    public void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned");
        } else {
            System.out.println("Book Not Issued");
        }
    }
    // Display Book
    public void display() {
        System.out.println("ID : " + id);
```

```
        System.out.println("Title : " + title);
        System.out.println("Status : " + (issued ? "Issued" : "Available"));
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book[] books = new Book[5];
        int count = 0, choice;
        do {
            System.out.println("\n1.Add Book");
            System.out.println("2.Display Books");
            System.out.println("3.Issue Book");
            System.out.println("4.Return Book");
            System.out.println("5.Search Book");
            System.out.println("6.Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Book ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();
                    System.out.print("Enter Book Title: ");
                    String title = sc.nextLine();
                    books[count] = new Book(id, title);
                    count++;
                    System.out.println("Book Added");
                    break;
                case 2:
                    for (int i = 0; i < count; i++)
                        books[i].display();
                    break;
                case 3:
                    System.out.print("Enter Book ID: ");
                    id = sc.nextInt();
```

```
                    id = sc.nextInt();
                    for (int i = 0; i < count; i++)
                        if (books[i].getId() == id)
                            books[i].issueBook();
                    break;
                case 4:
                    System.out.print("Enter Book ID: ");
                    id = sc.nextInt();
                    for (int i = 0; i < count; i++)
                        if (books[i].getId() == id)
                            books[i].returnBook();
                    break;
                case 5:
                    System.out.print("Enter Book ID: ");
                    id = sc.nextInt();
                    for (int i = 0; i < count; i++)
                        if (books[i].getId() == id)
                            books[i].display();
                    break;
                case 6:
                    System.out.println("Thank You");
                    break;
                default:
                    System.out.println("Invalid Choice");
            }
        } while (choice != 6);
    }
}
```

OUTPUT:

```
1.Add Book
2.Display Books
3.Issue Book
4.Return Book
5.Search Book
6.Exit
Enter Choice: 1
Enter Book ID: 101
Enter Book Title: java
Book Added

1.Add Book
2.Display Books
3.Issue Book
4.Return Book
5.Search Book
6.Exit
Enter Choice: 2
ID : 101
Title : java
Status : Available

1.Add Book
2.Display Books
3.Issue Book
4.Return Book
5.Search Book
6.Exit
Enter Choice: 3
Enter Book ID: 101
Book Issued

1.Add Book
2.Display Books
3.Issue Book
4.Return Book
5.Search Book
6.Exit
Enter Choice: 4
Enter Book ID: 101
Book Returned
```

```
1.Add Book
2.Display Books
3.Issue Book
4.Return Book
5.Search Book
6.Exit
Enter Choice: 5
Enter Book ID: 101
ID : 101
Title : java
Status : Available

1.Add Book
2.Display Books
3.Issue Book
4.Return Book
5.Search Book
6.Exit
Enter Choice: 6
Thank You


...Program finished with exit code 0
Press ENTER to exit console.
```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary

differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

```java
import java.util.Scanner;
class Employee {
    String name;
    Employee(String name) {
        this.name = name;
    }
    void calculateSalary() {
        System.out.println("Salary Calculation");
    }
}
class PermanentEmployee extends Employee {
    double basic;
    PermanentEmployee(String name, double basic) {
        super(name);
        this.basic = basic;
    }
    void calculateSalary() {
        double salary = basic + (0.2 * basic); // 20% bonus
        System.out.println("Employee : " + name);
        System.out.println("Salary : " + salary);
    }
}
class ContractEmployee extends Employee {
    int hours;
    double rate;
    ContractEmployee(String name, int hours, double rate) {
        super(name);
        this.hours = hours;
        this.rate = rate;
    }
    void calculateSalary() {
        double salary = hours * rate;
        System.out.println("Employee : " + name);
        System.out.println("Salary : " + salary);
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Permanent Employee");
        System.out.println("2. Contract Employee");
        System.out.print("Enter Choice: ");
        int choice = sc.nextInt();
        sc.nextLine();
        Employee emp;
        if (choice == 1) {
            System.out.print("Enter Name: ");
            String name = sc.nextLine();
            System.out.print("Enter Basic Salary: ");
            double basic = sc.nextDouble();
            emp = new PermanentEmployee(name, basic);
        } else {
            System.out.print("Enter Name: ");
            String name = sc.nextLine();
            System.out.print("Enter Hours Worked: ");
            int hours = sc.nextInt();
            System.out.print("Enter Rate Per Hour: ");
            double rate = sc.nextDouble();
            emp = new ContractEmployee(name, hours, rate);
        }
        emp.calculateSalary();
    }
}
```

OUTPUT:

```
1. Permanent Employee
2. Contract Employee
Enter Choice: 1
Enter Name: BHARATH
Enter Basic Salary: 200000
Employee : BHARATH
Salary : 240000.0


...Program finished with exit code 0
Press ENTER to exit console.
```

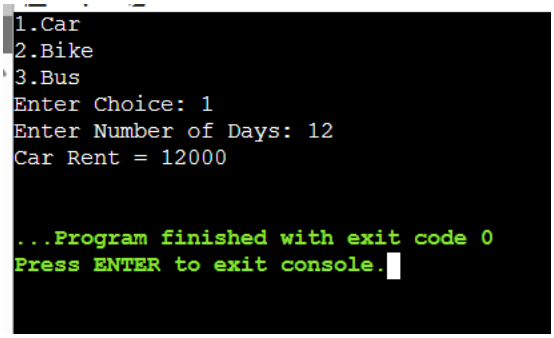
3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

```java
import java.util.Scanner;
class Vehicle {
    String name;
    Vehicle(String name) {
        this.name = name;
    }
    void calculateRent(int days) {
        System.out.println("Rent Calculation");
    }
}
class Car extends Vehicle {
    Car(String name) {
        super(name);
    }
    void calculateRent(int days) {
        System.out.println("Car Rent = " + (days * 1000));
    }
}
class Bike extends Vehicle {
    Bike(String name) {
        super(name);
    }
    void calculateRent(int days) {
        System.out.println("Bike Rent = " + (days * 500));
    }
}
class Bus extends Vehicle {
    Bus(String name) {
        super(name);
    }
    void calculateRent(int days) {
        System.out.println("Bus Rent = " + (days * 2000));
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        Vehicle[] v = new Vehicle[3];
        v[0] = new Car("Car");
        v[1] = new Bike("Bike");
        v[2] = new Bus("Bus");
        System.out.println("1.Car");
        System.out.println("2.Bike");
        System.out.println("3.Bus");
        System.out.print("Enter Choice: ");
        int choice = sc.nextInt();
        System.out.print("Enter Number of Days: ");
        int days = sc.nextInt();
        v[choice - 1].calculateRent(days);
    }
}
```

OUTPUT:



4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

```java
import java.util.Scanner;
class BankAccount {
    private int accountNumber;
    private String holderName;
    private double balance;
    BankAccount(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Amount Deposited");
        } else {
            System.out.println("Invalid Amount");
        }
    }
    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Amount Withdrawn");
        } else {
            System.out.println("Insufficient Balance");
        }
    }
    void showBalance() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Holder Name : " + holderName);
        System.out.println("Balance : " + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount acc = new BankAccount(101, "Rahul", 5000);
        System.out.println("1.Deposit");
        System.out.println("2.Withdraw");
        System.out.println("3.Show Balance");
        System.out.print("Enter Choice: ");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                System.out.print("Enter Amount: ");
                double dep = sc.nextDouble();
                acc.deposit(dep);
                acc.showBalance();
                break;
            case 2:
                System.out.print("Enter Amount: ");
                double wd = sc.nextDouble();
                acc.withdraw(wd);
                acc.showBalance();
                break;
            case 3:
                acc.showBalance();
                break;
            default:
                System.out.println("Invalid Choice");
        }
    }
}
```

OUTPUT:

```
1.Deposit
2.Withdraw
3.Show Balance
Enter Choice: 1
Enter Amount: 12000
Amount Deposited
Account Number : 101
Holder Name : Rahul
Balance : 17000.0


...Program finished with exit code 0
Press ENTER to exit console.
```

5. University Admission System using Method Overloading

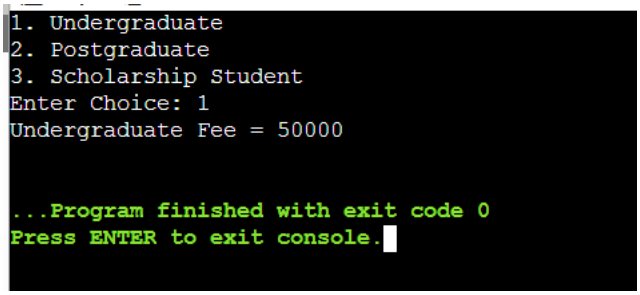
Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

```java
import java.util.Scanner;
class Admission {
    void calculateFee() {
        System.out.println("Undergraduate Fee = 50000");
    }
    void calculateFee(int pg) {
        System.out.println("Postgraduate Fee = 70000");
    }
    void calculateFee(double scholarship) {
        double fee = 50000 - scholarship;
        System.out.println("Scholarship Student Fee = " + fee);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Admission a = new Admission();
        System.out.println("1. Undergraduate");
        System.out.println("2. Postgraduate");
        System.out.println("3. Scholarship Student");
        System.out.print("Enter Choice: ");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                a.calculateFee();
                break;
            case 2:
                a.calculateFee(1);
                break;
            case 3:
                System.out.print("Enter Scholarship Amount: ");
                double amount = sc.nextDouble();
                a.calculateFee(amount);
                break;
            default:
                System.out.println("Invalid Choice");
```

OUTPUT:



6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

```java
import java.util.Scanner;
abstract class Shape {
    abstract void area();
}
class Circle extends Shape {
    double r;
    Circle(double r) {
        this.r = r;
    }
    void area() {
        System.out.println("Area of Circle = " + (3.14 * r * r));
    }
}
class Rectangle extends Shape {
    double l, b;
    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }
    void area() {
        System.out.println("Area of Rectangle = " + (l * b));
    }
}
class Triangle extends Shape {
    double base, height;
    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    void area() {
        System.out.println("Area of Triangle = " + (0.5 * base * height));
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

OUTPUT:

```
Enter Circle Radius: 24
Enter Rectangle Length and Breadth: 2 3
Enter Triangle Base and Height: 2 3

Areas:
Area of Circle = 1808.6399999999999
Area of Rectangle = 6.0
Area of Triangle = 3.0


...Program finished with exit code 0
Press ENTER to exit console.
```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

```java
import java.util.Scanner;

interface MedicalRecord {
    void addRecord();
    void displayRecord();
}

class Patient implements MedicalRecord {
    int id;
    String name;
    Scanner sc = new Scanner(System.in);

    public void addRecord() {
        System.out.print("Enter Patient ID: ");
        id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Patient Name: ");
        name = sc.nextLine();
    }

    public void displayRecord() {
        System.out.println("Patient ID : " + id);
        System.out.println("Patient Name : " + name);
    }
}

class Doctor implements MedicalRecord {
    int id;
    String name;
    Scanner sc = new Scanner(System.in);

    public void addRecord() {
        System.out.print("Enter Doctor ID: ");
        id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Doctor Name: ");
```

OUTPUT:

```
1. Patient
2. Doctor
Enter Choice: 2
Enter Doctor ID: 361
Enter Doctor Name: BHARATH SIMHA REDDY
Doctor ID : 361
Doctor Name : BHARATH SIMHA REDDY


...Program finished with exit code 0
Press ENTER to exit console.
```

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

```java
class Product {
    String name;
    double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}

class Order {
    void placeOrder(Product p, int qty) {
        double total = p.price * qty;

        System.out.println("Product : " + p.name);
        System.out.println("Price : " + p.price);
        System.out.println("Quantity : " + qty);
        System.out.println("Total Amount : " + total);
    }
}

public class Main {
    public static void main(String[] args) {

        Product p = new Product("Laptop", 50000);

        Order o = new Order();

        o.placeOrder(p, 2);
    }
}
```

OUTPUT:

```
Product : Laptop
Price : 50000.0
Quantity : 2
Total Amount : 100000.0


...Program finished with exit code 0
Press ENTER to exit console.
```

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

```java
import java.util.Scanner;
class Student {
    int roll;
    String name;
    int m1, m2, m3;
    int total;
    double average;
    char grade;
    Student(int roll, String name, int m1, int m2, int m3) {
        this.roll = roll;
        this.name = name;
        this.m1 = m1;
        this.m2 = m2;
        this.m3 = m3;
    }
    void calculate() {
        total = m1 + m2 + m3;
        average = total / 3.0;
        if (average >= 90)
            grade = 'A';
        else if (average >= 75)
            grade = 'B';
        else if (average >= 50)
            grade = 'C';
        else
            grade = 'F';
    }
    void display() {
        System.out.println("Roll No : " + roll);
        System.out.println("Name : " + name);
        System.out.println("Total : " + total);
        System.out.println("Average : " + average);
        System.out.println("Grade : " + grade);
        System.out.println();
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Number of Students: ");
        int n = sc.nextInt();
        Student[] s = new Student[n];
        for (int i = 0; i < n; i++) {
            System.out.println("\nStudent " + (i + 1));
            System.out.print("Roll No: ");
            int roll = sc.nextInt();
            sc.nextLine();
            System.out.print("Name: ");
            String name = sc.nextLine();
            System.out.print("Marks in 3 Subjects: ");
            int m1 = sc.nextInt();
            int m2 = sc.nextInt();
            int m3 = sc.nextInt();
            s[i] = new Student(roll, name, m1, m2, m3);
            s[i].calculate();
        }
        int max = s[0].total;
        int topper = 0;
        for (int i = 1; i < n; i++) {
            if (s[i].total > max) {
                max = s[i].total;
                topper = i;
            }
        }
        System.out.println("\nStudent Details");
        for (int i = 0; i < n; i++) {
            s[i].display();
        }
        System.out.println("Class Topper");
        System.out.println("Name : " + s[topper].name);
        System.out.println("Total : " + s[topper].total);
    }
```

OUTPUT:

```
Enter Number of Students: 2

Student 1
Roll No: 1
Name: BHARATH
Marks in 3 Subjects: 95 98 99

Student 2
Roll No: 2
Name: BALU
Marks in 3 Subjects: 95 95 95

Student Details
Roll No : 1
Name : BHARATH
Total : 292
Average : 97.33333333333333
Grade : A

Roll No : 2
Name : BALU
Total : 285
Average : 95.0
Grade : A

Class Topper
Name : BHARATH
Total : 292


...Program finished with exit code 0
Press ENTER to exit console.
```

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

```java
import java.util.Scanner;
class Device {
    String name;
    Device(String name) {
        this.name = name;
    }
    void turnOn() {
        System.out.println(name + " is ON");
    }
    void turnOff() {
        System.out.println(name + " is OFF");
    }
    void powerConsumption() {
        System.out.println("Power Consumption");
    }
}
class Light extends Device {
    Light() {
        super("Light");
    }
    void powerConsumption() {
        System.out.println("Power Consumption = 20 Watts");
    }
}
class Fan extends Device {
    Fan() {
        super("Fan");
    }
    void powerConsumption() {
        System.out.println("Power Consumption = 75 Watts");
    }
}
class AirConditioner extends Device {
    AirConditioner() {
        super("Air Conditioner");
    }
```

OUTPUT:

```
1. Light
2. Fan
3. Air Conditioner
Enter Choice: 1
Light is ON
Power Consumption = 20 Watts
Light is OFF


...Program finished with exit code 0
Press ENTER to exit console.
```